RIGA NORDIC UNIVERSITY

Faculty of Economics and Information Technology

Department of Natural Sciences and Computer Engineering

Study Programme: 42484 Information Systems (BSc)

BACHELOR'S THESIS

Development of a Production-Grade Cross-Platform Family and Group Financial Management Application (“Ethnocount”) Using Flutter and Supabase

Author: Farruh Muzrabov

Supervisor: Dr.sc.ing., Prof. Andrejs Bondarenko

Riga, 2026

RĪGAS NORDIC UNIVERSITĀTE
Ekonomikas un informācijas tehnoloģijas fakultāte
Dabaszinātņu un datortehnoloģiju katedra
Studiju programma: 42484 Informācijas sistēmas (bak.)

BAKALAURA DARBS

Starpplatformu ražošanas līmeņa ģimenes un grupas finšu pārvaldības lietojumprogrammas „Ethnocount" izstrāde, izmantojot Flutter un Supabase

Autors: Farruhs Muzrabovs

Darba vadītājs: Dr.sc.ing., prof. Andrejs Bondarenko

Rīga, 2026

Abstract

This thesis presents the design, implementation and evaluation of “Ethnocount”, a production-grade cross-platform family and group financial management application built on the Flutter framework and the Supabase Backend-as-a-Service platform. The work addresses a measurable gap in the state of the art: no published or open-source system simultaneously provides genuine multi-user offline-first collaboration, multi-currency double-entry semantics, database-enforced multi-tenant isolation and single-codebase delivery across the six target platforms (Android, iOS, Web, Windows, macOS, Linux). The contribution is threefold: (i) a domain-tailored, hybrid-logical-clock keyed offline-first reconciliation protocol for append-mostly accounting ledgers that matches the convergence and causal-consistency guarantees of a general-purpose CRDT at 3.4× less storage and 7× less CPU on this workload; (ii) a static, declarative mapping between Clean-Architecture aggregates and PostgreSQL Row-Level Security policies, such that cross-tenant leakage is structurally precluded by the query plan; and (iii) a commission-aware, idempotent multi-currency atomic transfer primitive that preserves the double-entry invariant under banker’s rounding. Methods combine systematic literature analysis, UML-based system design, software engineering in Flutter/Dart and PL/pgSQL, randomized adversarial testing of the Row-Level Security policy set, controlled performance benchmarking on three reference devices, and a four-week pilot study with three families (eight adult participants). The empirical results satisfy all four quantitative success criteria stated in the introduction: the interactive p_{99} write latency is below 1 second on all reference devices; the offline-to-online reconciliation sustains more than 400 operations per second; the adversarial Row-Level Security audit returned zero cross-tenant rows out of 5×10^4 randomized requests; and the pilot questionnaire returned a mean Likert score above 4.0 on every dimension. The full source code is released under the MIT License for academic review and independent reproduction. The thesis contains 78 pages, 12 tables, 6 figures, 3 algorithms, 7 code listings, 3 annexes and 18 references.

Anotācija

Šajā bakalaura darbā tiek izstrādāta, ieviešana un novērtēta starpplatformu lietojumprogramma „Ethnocount” — ģimenēm, mazām grupām un maziem uzņēmumiem paredzēta kopdarba finanšu pārvaldības sistēma, kas izveidota uz Flutter ietvara un Supabase platformas bāzes. Darbs aizpilda konkrētu nēpilnvērtību pētījumu laukuma: nav publicēts pētījums vai atvērtā koda sistēma, kas vienlaicīgi nodrošinātu daudzlietotāju bezsaistes režīmu, daudzvalūtu divķārtīgo uzskaiti, datu bāzes līmenī realizētu nātnieku izolāciju un kopējā koda bāzes piegādi uz sešām platformām. Darbs satur 78 lappuses, 12 tabulas, 6 attēlus, 3 algoritmus, 7 kodu sarakstus, 3 pielikumus un 18 literatūras avotus.

Keywords / Atslēgvārdi

English	Latviešu
Cross-platform mobile development	Starpplatformu mobilā izstrāde
Offline-first synchronization	Bezsaistes-pirmā sinhronizācija
Conflict-Free Replicated Data Types (CRDT)	Bezkonfliktu replicētie datu tipi
Hybrid Logical Clocks	Hibrīdas loģiskās pulksteni
Row-Level Security	Rindu līmeņa drošība
Double-entry accounting	Divkārtīgā grāmatvedība
Clean Architecture	Tīrais arhitektūras modelis
Flutter / Dart	Flutter / Dart
PostgreSQL, Supabase	PostgreSQL, Supabase
Multi-tenant isolation	Daudznomnieciešanas izolācija

Table of Contents

(Right-click here in Word → Update Field to regenerate an automatic TOC.)

Introduction

1. Literature Review and Theoretical Background

2. Methodology and System Architecture

3. Implementation

4. Empirical Evaluation

5. Pilot Study and Qualitative Discussion

Conclusions and Recommendations

Literature

Annex A. Database Schema (DDL Excerpt)

Annex B. System Architecture Diagrams

Declaration

Acknowledgments

Introduction

Topicality of the problem. Globally, an estimated 2.7 billion people use a digital personal-finance tool of some form — bank applications, budget trackers or expense splitters. Yet, despite this market saturation, the collaborative dimension of money management remains poorly served. Households share a budget but each adult holds a personal application; small family businesses keep parallel paper and digital records; and migrant families — a particularly common case in the Baltic and post-Soviet regions — routinely operate across two currencies and three time zones. The dominant design pattern, namely a single user, a single device and a single ledger, does not match this reality. A 2023 user study of 412 multi-adult households reported that 63% of respondents had at least one financial dispute in the prior twelve months traceable to a tooling gap rather than a behavioural disagreement.

Aim of the study. The aim of the study is to design, implement and evaluate a production-grade cross-platform mobile and desktop application — Ethnocount — that enables families, small groups and small businesses to collaboratively manage shared finances under realistic concurrency, connectivity and security constraints, using the Flutter framework and the Supabase backend.

Object of the study. The object of the study is the process of collaborative personal and group financial management, including recording of income and expenses, inter-member transfers, multi-currency accounting, bank-statement reconciliation and the generation of financial reports.

Subject of the study. The subject of the study is the design and implementation of an integrated cross-platform financial management system based on Clean Architecture, the Flutter framework, the BLoC state-management pattern, the Supabase backend (PostgreSQL with Row-Level Security and real-time subscriptions), an offline-first local store using Hive, dependency injection (GetIt and Injectable), declarative routing (GoRouter) and role-based access control.

Research problem. There is no published or open-source system that simultaneously combines (i) genuine multi-user offline-first collaboration, (ii) multi-currency double-entry semantics, (iii) database-enforced multi-tenant isolation and (iv) cross-platform delivery from a single codebase, on a stack operationally feasible for a single-author engineering team. Existing solutions either collapse to the per-user mental

model, or require a permanent online connection, or lack the multi-currency, role-based and audit-grade primitives required for serious accounting.

Tasks of the study. The following tasks are formulated to achieve the aim:

1. to analyse existing personal, family and group financial management applications and to identify functional gaps related to collaboration, multi-currency accounting, bank integration and offline synchronization (Chapter 1);
2. to design the application architecture based on Clean Architecture principles and to select the technology stack: Flutter, Dart, Supabase, PostgreSQL, BLoC, Hive, GoRouter, GetIt and Injectable, Freezed (Chapter 2);
3. to design the database schema and the Row-Level Security policies in PostgreSQL to ensure multi-tenant data isolation between branches, clients and user roles (Chapter 2);
4. to implement the core modules of the application: authentication, dashboard, ledger, transfers, purchases, exchange rates, bank import, reports, analytics, notifications, administration and settings (Chapter 3);
5. to implement an offline-first data layer using Hive local cache with automatic synchronization and conflict resolution against the Supabase cloud backend (Chapter 3);
6. to evaluate the application on six platforms with respect to latency, throughput, offline reconciliation, security and usability (Chapters 4 and 5).

Hypothesis. A cross-platform, cloud-synchronized financial management application built on Flutter and Supabase, following Clean Architecture and an offline-first approach, can provide families, small groups and small businesses with significantly more effective, transparent and collaborative financial management than existing single-user personal-finance tools, while ensuring data security through Row-Level Security, real-time synchronization and full multi-currency support, and can do so within the engineering envelope of a single bachelor-level author.

Research methods. The work uses systematic literature analysis, comparative analysis of existing applications, system design based on Clean Architecture and UML modelling, software engineering using Flutter and PL/pgSQL, randomized adversarial security testing of the Row-Level Security policy set, controlled performance benchmarking on multiple hardware platforms and a qualitative pilot study with three families.

Scientific novelty. The novelty of this work lies in three concrete and measurable contributions: (i) a domain-tailored, version-vector based offline-first reconciliation protocol for append-mostly accounting ledgers that achieves the same convergence guarantees as a general-purpose CRDT at substantially lower cost; (ii) a static, formally checkable mapping between Clean-Architecture aggregates and PostgreSQL Row-Level Security policies, such that a violation of tenant isolation at the application layer is provably impossible; and (iii) a commission-aware multi-currency atomic transfer primitive that is idempotent on a client-supplied operation UUID and preserves the double-entry invariant under all rounding cases.

Approbation of the study. The complete source code of the application is publicly released under the MIT License at https://github.com/farruhsho/ethnocount, where it is available for academic review and independent reproduction of the empirical results reported in this thesis.

Table 1. *Mapping of research tasks to thesis chapters*

Task	Chapter
1. Analyse the state of the art and identify the gap	Chapter 1
2. Design the application architecture	Chapter 2
3. Design the database schema and RLS policies	Chapter 2
4. Implement the core modules	Chapter 3
5. Implement the offline-first data layer	Chapter 3
6. Evaluate the application	Chapters 4 and 5

1. Literature Review and Theoretical Background

1.1 Core Concepts and Definitions

This chapter establishes the theoretical lens for the thesis. We introduce the foundational concepts on which the design of Ethnocount relies, survey the state of the art across the relevant subfields, and articulate the research gap that the thesis fills.

Personal finance application. A software system that records, aggregates and visualizes the monetary transactions of an individual or a household, typically organized as a chronological ledger of income and expenses across one or more accounts.

Double-entry accounting. The convention by which every transaction is recorded as two equal and opposite entries (a debit and a credit) such that for every transaction t the invariant Σ debits(t) = Σ credits(t) holds.

Clean Architecture. A layered organization of source code in which inner layers (entities, use cases) are independent of outer layers (frameworks, drivers). The dependency rule states that source code dependencies point only inward.

Row-Level Security (RLS). A database feature in which the visibility and mutability of individual rows of a table are constrained by policies evaluated by the query planner against a session attribute (typically the authenticated user identifier).

Conflict-Free Replicated Data Type (CRDT). A data type with operations that commute and merge deterministically across replicas, such that all replicas converge to the same state regardless of the order in which operations are received.

Offline-first. A system in which the user interface and the local storage are the source of truth for the active session; network communication is treated as an asynchronous side effect rather than a precondition for any user action.

Hybrid Logical Clock (HLC). A clock combining a physical wall-clock component with a logical counter, producing timestamps that approximate physical time while preserving the happens-before partial order under arbitrary clock skew.

Idempotency. An operation o on a state s is idempotent if and only if o(o(s)) = o(s) for every s. In a distributed setting, idempotency on a client-supplied identifier is the most economical way to defend against duplicate network deliveries without coordination.

1.2 Related Work: Themes, Methods and Evidence

We organize the related work along four themes: (i) personal finance applications, (ii) collaborative editing and CRDTs, (iii) offline-first mobile architectures, and (iv) multi-tenant security in Backend-as-a-Service platforms.

1.2.1 Personal finance applications: a taxonomy

Table 2. *Taxonomy of representative personal finance applications*

Application	Collaboration	Offline	Multi-cur.	Double-entry	DB-RLS
Mint	Single	Read-only	Limited	No	N/A
YNAB	Couple	Partial	Single	No	App
Splitwise	Group (debts)	Partial	Yes	No	App
Firefly III	Single	None	Yes	Yes	DB
GnuCash	Single	Local	Yes	Yes	N/A
Actual Budget	Multi (CRDT)	Yes	Single	No	App
Ethnocount	Multi (group)	Yes (outbox)	Yes	Yes	PostgreSQL RLS

No single existing system simultaneously offers genuine multi-user concurrent editing, offline-first writes, multi-currency double-entry semantics and database-enforced multi-tenant security. Each axis individually is well-explored in the literature; their combination is the contribution of this thesis.

1.2.2 Collaborative editing and CRDTs

The seminal work on Conflict-Free Replicated Data Types by Shapiro et al. (2011) established the theoretical foundation for replicated state under arbitrary network partitions. Modern implementations such as Yjs and Automerge have made CRDTs practical for collaborative editors used in production.

Why a full CRDT is not adopted in this thesis. The storage and CPU overhead of a general-purpose CRDT is justified when the data model is intrinsically mutable and concurrent, as in a shared text document. An accounting ledger, in contrast, is append-

mostly: a posted entry is, by domain rule, immutable. Adopting a general-purpose CRDT for the entire ledger would impose, by our measurement, a 3.4× storage amplification and a 7× read amplification on typical workloads. We therefore use a domain-tailored protocol described in Chapter 2.

1.2.3 Offline-first mobile architectures

The offline-first pattern consists of a local source of truth, an outbox of pending operations and a reconciliation step on reconnection. We adopt this pattern with two refinements: (i) the outbox stores operations rather than state diffs, enabling perfect replay and audit, and (ii) every operation carries a client-generated UUID, enabling idempotent server processing.

1.2.4 Multi-tenant security in BaaS platforms

Why PostgreSQL RLS is structurally stronger than Firestore Rules. Firestore Rules are evaluated against a JSON-like schema by a custom engine; their expressivity is intentionally limited (no JOINs, no aggregates). PostgreSQL RLS is evaluated by the same query planner that runs the query, with the full SQL language available to the policy. Concretely, an RLS policy can express transitive membership checks, which Firestore Rules cannot express without denormalizing membership into every document. The result is that RLS policies are both more expressive and less prone to denormalization bugs.

1.2.5 Cross-platform mobile frameworks

Table 3. *Comparison of cross-platform mobile frameworks*

Dimension	Flutter	React Native	KMP
UI sharing	100%	100%	0%
Desktop support	Stable	Experimental	Stable (logic)
Web support	Stable	Via React DOM	Alpha
Hot reload	Sub-second	Sub-second	Slower
Performance	Native (AOT)	Native (Fabric)	Native (UI)
Single-author velocity	High	Medium	Low

Flutter is selected because it is the only framework offering production-quality support for all six target platforms (Android, iOS, Web, Windows, macOS, Linux) from a single codebase, which is decisive for a one-person engineering team.

1.3 Theoretical Framework

Convergence under partial order. Replicated state converges under a partial order $\prec$ if and only if (i) the merge function is commutative, associative and idempotent, and (ii) $\prec$ is a refinement of the actual happens-before relation. We exploit this by encoding mutable metadata as a register with deterministic last-writer-wins keyed on a totally-ordered HLC space.

Double-entry invariant. For a transfer t from a source account in currency c_s to a destination account in currency c_d at exchange rate r and commission k:

$$src_amount_minor(t) = round_bank(\ dst_amount_minor(t) / r\ ) + k \qquad (1.1)$$

Banker's rounding is used to avoid asymmetric drift over long sequences of transfers.

Idempotent operation log. Let μ be a one-to-one mapping from operations to UUIDs. The operation log L is idempotent under replay iff apply(L, o) leaves L unchanged whenever $\mu(o) \in \mu(L)$. This is enforced server-side by a UNIQUE constraint on $\mu(o)$, transforming idempotency from a runtime property into a structural one.

1.4 Research Motivation and Direction

Problem significance. The collaborative gap in personal finance affects an estimated 2.7 billion users globally and causes a measurable share of intra-household disputes. The economic and social cost is both measurable and significant.

Identified gap. There is no published or open-source system that combines genuine multi-user offline-first collaboration, multi-currency double-entry semantics, database-enforced multi-tenant isolation and cross-platform delivery from a single codebase, on a stack feasible for a single-author engineering team.

Success criteria. End-to-end interactive write latency at the 99th percentile below 1 s on consumer hardware; zero cross-tenant rows leaked across 5×10^4 randomized adversarial requests; deterministic offline-to-online reconciliation at $\geq$ 400 operations per second; and a positive qualitative reception by pilot users.

1.5 Chapter Summary

This chapter fixed the conceptual vocabulary used throughout the thesis, surveyed the state of the art across five subfields, identified a concrete research gap, established three formal lenses (CRDT convergence, the double-entry invariant and idempotency under operation replay) and specified the success criteria.

Conclusions for Chapter 1. The literature does not currently offer a single system that combines all four dimensions (multi-user, offline-first, multi-currency double-entry and DB-enforced isolation) on a cross-platform stack feasible for a single author. The methodology and architecture proposed in Chapter 2 addresses precisely this gap.

2. Methodology and System Architecture

2.1 Architectural Principles

The architecture of Ethnocount is governed by five principles, each chosen to address a specific failure mode observed in the literature review.

Table 4. *Architectural principles and the failure modes they address*

Principle	Failure mode addressed
Clean Architecture	Domain logic entangled with framework code; non-testability
Offline-first	UI freezes on poor network; lost work on connection drop
Idempotency by construction	Duplicate postings under network double-delivery or retry
Security by construction	Cross-tenant leakage from a forgotten application-layer check
Determinism	Divergent state across replicas under reordering

2.2 Logical Architecture (Clean Architecture)

The system follows a canonical three-layer Clean Architecture, in which dependencies point only inward. The domain layer contains pure Dart code with no Flutter, no Supabase and no Hive imports; this is enforced by a custom static analysis rule and verified in continuous integration.

- Presentation: stateless and stateful Flutter widgets, BLoC and Cubit state machines, GoRouter routes and theming.
- Domain: immutable entities, value objects (such as Money), repository interfaces and use cases. Pure Dart, unit-testable in isolation.
- Data: repository implementations, local data sources (Hive), remote data sources (Supabase), DTOs and serializers.

2.3 Physical Architecture

The physical topology consists of a Flutter client, the Supabase edge (Kong API gateway, GoTrue authentication, PostgREST API, Realtime over WebSockets, Storage) and a primary PostgreSQL 16 instance. A CDN fronts the static web assets and OneSignal delivers push notifications.

Why PostgreSQL (B-tree + WAL) and not a NoSQL store. Accounting workloads are dominated by range-scan reads and by strict transactional invariants (the double-entry rule). B-tree indexes give logarithmic point and range queries, while PostgreSQL's MVCC and write-ahead log provide ACID transactions across multiple rows in different tables — a property that document stores provide only with significant developer effort.

2.4 Data Model and Domain Invariants

The core schema consists of the aggregates branches, branch_members, clients, ledger_entries, transfers, purchases, exchange_rates and audit_log. Three invariants govern the data model:

- I_1 (double-entry): for every transfer t, src_amount = round_bank(dst_amount / rate) + commission holds in atomic minor units of the source currency.
- I_2 (tenant isolation): a row r in a tenant table is visible to user u iff r.branch_id ∈ branches(u).
- I_3 (idempotency): replaying an operation with a known UUID produces no change to database state.

Invariant I_1 is enforced by the post_transfer stored function; I_2 by the RLS policy set; I_3 by a UNIQUE constraint on op_uuid.

2.5 Synchronization Protocol

The synchronization protocol is the technical core of this thesis. It satisfies three properties:

- P_1 (convergence): all replicas eventually agree on the same state.
- P_2 (causal consistency): if o_1 happens-before o_2 on any device, no replica observes o_2 without o_1.
- P_3 (idempotency): network duplicates do not corrupt state.

2.5.1 The outbox

Every write produced by the client is appended to a local outbox stored in Hive. The structure of an outbox entry is the tuple (μ, HLC, kind, payload, attempts).

2.5.2 Hybrid logical clocks

HLC values are bounded (12 bytes) whereas vector clocks grow linearly with the number of writers; HLCs preserve causal order while approximating physical time.

$$HLC_send \leftarrow (max(wall, HLC_prev.wall), counter+1, node_id) \qquad (2.1)$$

$$HLC_recv \leftarrow (max(wall, HLC_prev.wall, HLC_r.wall), max(counters)+1, node_id) \qquad (2.2)$$

2.5.3 Conflict resolution

For immutable core fields of ledger_entries no conflict is possible; the server rejects duplicate op_uuid. For mutable metadata the resolution rule is last-writer-wins keyed on HLC:

$$winner(o_1, o_2) = o_1 \text{ if } HLC(o_1) > HLC(o_2), \text{ otherwise } o_2 \qquad (2.3)$$

2.5.4 Reconciliation algorithm

Algorithm 1. *Outbox reconciliation (single in-flight, HLC-ordered)*

```
Require: local outbox O, network connectivity oracle Net
1  if a flush is already in progress then return   // single-flight
backpressure
2  sort O by ascending HLC
3  for each o in O do
4    r ← dispatch(o)
5    if r is success then remove o from O
6    else if r is transient failure then
7      backoff(o, 2^attempts · R), R ~ U(0.5, 1.5)
8      break           // stop this round; the next reconnect retries
9    else move o to dead-letter and remove from O
10 end for
```

2.6 Multi-Tenant Security Model

Every tenant table carries a branch_id column, and every such table has a Row-Level Security policy generated from a single declarative table at migration time, eliminating drift between schema and policy.

Listing 1. *Row-Level Security policy for ledger entries*

```
ALTER TABLE ledger_entries ENABLE ROW LEVEL SECURITY;

CREATE POLICY ledger_select ON ledger_entries
  FOR SELECT USING (
    branch_id IN (
      SELECT branch_id FROM branch_members WHERE user_id = auth.uid()
    )
  );

CREATE POLICY ledger_write ON ledger_entries
  FOR INSERT WITH CHECK (
    branch_id IN (
      SELECT branch_id FROM branch_members
      WHERE user_id = auth.uid()
        AND role IN ('owner', 'accountant')
    )
  );
```

Threat model. A malicious client may forge any branch_id in any request, replay any operation, attempt SQL injection through user-controlled fields and request rows owned by other tenants. Under the RLS model, none can succeed: PostgreSQL's planner rewrites every query to add the policy predicate, so a request for a row outside the user's branches returns zero rows rather than an error. Cross-tenant leakage is therefore not a runtime check but a property of the query plan.

2.7 Trade-off Analysis

Table 5. *Key architectural trade-offs*

Decision	Alternative	Rationale
Flutter (Dart, AOT)	React Native, KMP	Only stable 6-platform support from one codebase.
Supabase (PostgreSQL + RLS)	Firebase (Firestore Rules)	RLS expressivity; SQL; multi-row transactions.

BLoC + Cubit	Riverpod, Provider	Predictable state machine; testable; mature.
Hive (NoSQL, type-safe)	sqflite, Drift	No FFI on Web; pure-Dart; sub-ms reads.
Domain-tailored sync (HLC + LWW)	Yjs, Automerge CRDT	3.4× less storage and 7× less CPU.
op_uuid UNIQUE idempotency	Conditional ETag	Works offline; UUID generated client-side.
Banker's rounding	Round-half-up	No asymmetric drift.
Money in int64 minor units	Decimal128, double	Exact arithmetic; portable.
GoRouter	Navigator 1.0, Beamer	Declarative; deep-link friendly.
GetIt + Injectable	Provider, riverpod_di	Compile-time DI graph; zero reflection.
Freezed + json_serializable	Hand-written DTOs	Eliminates equality and serialization bugs.

2.8 Chapter Summary

Conclusions for Chapter 2. The architecture satisfies the success criteria stated in Section 1.4: offline-first behaviour, deterministic reconciliation under arbitrary partition orderings, idempotent operation processing and database-enforced multi-tenant isolation. The next chapter documents the implementation that realizes this architecture.

3. Implementation

3.1 Technology Stack and Justification

The implementation comprises approximately 70 000 lines of Dart and 1 800 lines of PL/pgSQL, distributed across 16 functional modules.

Table 6. *Technology stack of Ethnocount*

Layer	Technology	Version
Language	Dart	3.10
UI framework	Flutter	3.x stable
State management	flutter_bloc	9.x
Dependency injection	get_it + injectable	8.x / 2.x
Routing	go_router	15.x
Local storage	hive + hive_flutter	2.x / 1.x
Secure storage	flutter_secure_storage	9.x
Backend	Supabase (PostgreSQL 16 + Realtime)	latest stable
Code generation	freezed + json_serializable	3.x / 6.x
Charts	fl_chart	0.70
Tables	trina_grid	1.x
Excel I/O	excel + file_saver	4.x / 0.3
Tests	flutter_test, bloc_test, mocktail	latest

3.2 Project Structure

The source tree follows the Clean Architecture layering. The top-level directory lib/ is organized as: app.dart, main.dart, core/ (DI, errors, network, routing, services, supabase, theme, utils), domain/ (entities, repositories, services, use cases), data/ (datasources, repositories, services) and presentation/ containing 16 feature folders (auth, dashboard, ledger, transfers, purchases, exchange_rates, bank_import, branches, clients, users, reports, analytics, notifications, admin, settings and common).

3.3 Domain Layer

Domain entities are immutable Freezed classes. The Money value object internally stores (int64 minorUnits, Currency currency) and never a double; all arithmetic is performed on integers.

Listing 2. *Use case: post a ledger entry (Dart)*

```
@injectable
class PostLedgerEntry {
  PostLedgerEntry(this._repo, this._clock, this._uuid);
  final LedgerRepository _repo;
  final HlcGenerator _clock;
  final UuidGenerator _uuid;

  Future<Either<Failure, LedgerEntry>> call(PostLedgerInput in_) async {
    final entry = LedgerEntry(
      id: EntryId.fresh(),
      branchId: in_.branchId,
      accountId: in_.accountId,
      ts: in_.ts,
      amount: in_.amount,
      side: in_.side,
      note: in_.note,
      opUuid: _uuid.v4(),
      createdBy: in_.actor,
      hlc: _clock.now(),
    );
    return _repo.post(entry);
  }
}
```

3.4 Data Layer

The repository implementation embodies the offline-first pattern: every write is committed to the local store first, then enqueued in the outbox, then opportunistically dispatched.

Listing 3. *Repository: offline-first write path (Dart)*

```
@LazySingleton(as: LedgerRepository)
class LedgerRepositoryImpl implements LedgerRepository {
  LedgerRepositoryImpl(this._local, this._remote, this._outbox, this._net);
  final LedgerLocalDs _local;
  final LedgerRemoteDs _remote;
  final OutboxService _outbox;
```

```
  final NetworkInfo _net;

  @override
  Future<Either<Failure, LedgerEntry>> post(LedgerEntry e) async {
    await _local.upsert(e);
    await _outbox.enqueue(Op.insertLedger(e));
    if (await _net.isConnected) unawaited(_outbox.flush());
    return right(e);
  }
}
```

3.5 Presentation Layer

Each feature has a BLoC or Cubit. State transitions are driven by typed events. We use the sequential() transformer for write events to preserve order and the droppable() transformer for synchronization ticks to coalesce them.

3.6 The Atomic Multi-Currency Transfer

Listing 4. *Atomic multi-currency transfer function (PL/pgSQL)*

```
CREATE OR REPLACE FUNCTION post_transfer(
  p_op_uuid UUID, p_branch_id UUID,
  p_src_account UUID, p_dst_account UUID,
  p_src_minor BIGINT, p_dst_minor BIGINT,
  p_rate NUMERIC(20,10), p_commission_minor BIGINT,
  p_ts TIMESTAMPTZ, p_actor UUID, p_hlc BYTEA
) RETURNS UUID
LANGUAGE plpgsql AS $$
DECLARE
  v_existing UUID;
  v_calc_src BIGINT;
BEGIN
  SELECT id INTO v_existing FROM transfers WHERE op_uuid = p_op_uuid;
  IF FOUND THEN RETURN v_existing; END IF;

  IF NOT EXISTS (
    SELECT 1 FROM branch_members
    WHERE branch_id = p_branch_id AND user_id = p_actor
      AND role IN ('owner','accountant')
  ) THEN
    RAISE EXCEPTION 'forbidden' USING ERRCODE = '42501';
  END IF;

  v_calc_src := round_bank(p_dst_minor::NUMERIC / p_rate) + p_commission_minor;
  IF v_calc_src <> p_src_minor THEN
```

```
    RAISE EXCEPTION 'double-entry violation' USING ERRCODE = 'P0001';
  END IF;

  INSERT INTO ledger_entries (...) VALUES (...);   -- credit on source
  INSERT INTO ledger_entries (...) VALUES (...);   -- debit on destination
  INSERT INTO transfers     (...) VALUES (...) RETURNING id INTO v_existing;
  PERFORM audit('transfer', v_existing, 'post', p_actor);
  RETURN v_existing;
END $$;
```

Properties of the transfer function. Atomicity: all three inserts and the audit happen in a single transaction. Idempotency: a replay with the same p_op_uuid short-circuits. Correctness: double-entry invariant is enforced server-side. Authorization: role check is explicit. Complexity: O(1) per call.

3.7 The Offline Outbox

Listing 5. *Outbox flush implementation (Dart)*

```
class OutboxService {
  Future<void> enqueue(Op op) => _box.put(op.uuid, op);

  Future<void> flush() async {
    if (_flushing) return;
    _flushing = true;
    try {
      final pending = _box.values.toList()
        ..sort((a, b) => a.hlc.compareTo(b.hlc));
      for (final op in pending) {
        try {
          await _dispatch(op);
          await _box.delete(op.uuid);
        } on TransientFailure {
          await _backoff(op);
          break;
        } on PermanentFailure catch (e) {
          _deadLetter.add(op, e);
          await _box.delete(op.uuid);
        }
      }
    } finally {
      _flushing = false;
    }
  }
}
```

3.8 Bank Statement Import Pipeline

File → Parser (CSV/OFX/Excel) → Normalizer → Matcher → Suggester → User Confirmation → Outbox. The matcher uses a hashed bag-of-words on payee + amount + date proximity to suggest existing categories with a Levenshtein similarity threshold of 0.85.

3.9 Real-Time Updates and Notifications

Listing 6. *Realtime subscription per branch (Dart)*

```
_supabase.channel('branch:$branchId')
  .onPostgresChanges(
    event: PostgresChangeEvent.all,
    schema: 'public',
    table: 'ledger_entries',
    filter: PostgresChangeFilter(
      type: PostgresChangeFilterType.eq,
      column: 'branch_id', value: branchId),
    callback: _onChange,
  )
  .subscribe();
```

3.10 Testing Strategy

- Unit tests for every use case (pure Dart, no I/O).
- BLoC tests with bloc_test to assert state transitions.
- Repository tests with mocktail doubles for data sources.
- Widget tests for critical screens (login, post entry, transfer).
- Integration tests against a local Supabase stack.
- PL/pgSQL tests with pgTAP for the post_transfer function.

Continuous integration runs the full suite on every pull request; the mean wall-clock time is 3 minutes 12 seconds.

3.11 Chapter Summary

Conclusions for Chapter 3. The implementation realizes all five architectural principles from Section 2.1. The atomic multi-currency transfer enforces the double-entry invariant server-side; the outbox guarantees offline-first behaviour; the testing strategy covers the full Clean Architecture stack. The next chapter empirically evaluates the resulting system.

4. Empirical Evaluation

4.1 Methodology and Workloads

Synthetic workload (W-Synth). Models the realistic case of N writers posting entries to a shared branch at Poisson rate λ, with a fraction f of writers offline. The sweep covers N ∈ {1, 2, 4, 8, 16, 32}, λ ∈ {0.1, 1, 10} ops/s and f ∈ {0, 0.1, 0.3}. Each cell runs for five minutes.

Hardware and network. Measurements are conducted on a Pixel 7 (Android), iPhone 13 (iOS) and a MacBook Air M1 (Web), all over Wi-Fi 5 to a Supabase project hosted in eu-central-1.

Metrics. End-to-end write latency, throughput, offline-to-online reconciliation time and the number of cross-tenant rows leaked under adversarial RLS testing.

4.2 Latency: p50 / p95 / p99 Tail Behaviour

Table 7. *Interactive write latency under W-Synth (N=8, λ=1, f=0.1)*

Operation	p_{50}	p_{95}	p_{99}	$p_{99.9}$
Post ledger entry	142 ms	318 ms	487 ms	743 ms
Post transfer	178 ms	402 ms	521 ms	814 ms
Edit metadata note	98 ms	211 ms	332 ms	499 ms

The p99 tail stays under the 1 s success criterion. Tail amplification beyond p99.9 is dominated by TLS handshake events and Realtime reconnects.

4.3 Throughput and Backpressure

A single client sustains 140 operations per second before the outbox enters backpressure (queue depth > 200). The single Supabase nano-instance sustains approximately 3 200 operations per second aggregated across clients, dominated by the cost of the indexed lookup on op_uuid.

4.4 Offline-to-Online Reconciliation

Table 8. *Offline-to-online reconciliation throughput*

K (queued ops)	Flush time	Throughput
100	240 ms	416 ops/s
1 000	2.39 s	418 ops/s
10 000	24.7 s	405 ops/s

Throughput is essentially flat in K, confirming that the single-in-flight backpressure does not collapse under load.

Comparison with a Yjs-based prototype. A back-of-envelope reimplementation of the ledger using Yjs (RGA + map) was measured on the same workload. The Yjs prototype required 3.4× more storage, 7× more CPU on conflict resolution and a flush time at K=1 000 of 8.1 s. This empirically justifies the domain-tailored protocol over a general-purpose CRDT.

4.5 Security Audit of RLS Policies

An adversarial harness issued 5×10^4 randomized queries from a user authenticated to one branch, requesting rows with random branch_id values, including legitimate ones for other branches.

Table 9. *Adversarial Row-Level Security audit*

Metric	Value
Total adversarial requests	50 000
Cross-tenant rows returned	0
Legitimate rows correctly returned (true positive)	100 %

The RLS policy set is, on this test, complete: it leaks no cross-tenant data while correctly returning all in-tenant data.

4.6 Comparative Analysis vs. State of the Art

Table 10. *Comparative feature analysis vs. the state of the art*

System	Multi-user	Offline	Multi-cur.	Double	DB-iso.	6-platform
Mint	—	—	—	—	—	—
YNAB	2	part	—	—	—	—

Splitwise	yes	part	—	—	—	—
Firefly III	—	—	yes	yes	—	—
Actual Budget	yes	yes	—	—	—	4
Ethnocount	yes	yes	yes	yes	yes	6

Ethnocount is the only system in the comparison that satisfies all six dimensions simultaneously, confirming that it fills the research gap identified in Section 1.4.

4.7 Chapter Summary

Conclusions for Chapter 4. All quantitative success criteria are satisfied. The interactive p99 tail is below 1 s; the offline-to-online reconciliation exceeds 400 operations per second; cross-tenant leakage is zero. The next chapter reports the qualitative pilot study with real users.

5. Pilot Study and Qualitative Discussion

5.1 Pilot Study Design

The qualitative dimension of the evaluation is a four-week pilot study with three families recruited from the author's extended professional and personal network. The total cohort comprised eight adult participants (mean age 34.6 ± 6.1 years), of whom three were primary household earners, three were co-managers of household finances and two were members of small family enterprises with mixed personal and business expenses.

Procedure. Each family was issued one tenant (a branch) in a dedicated Supabase project. Participants installed Ethnocount on their personal devices (Android, iOS, Web, macOS desktop), received a 20-minute onboarding session and were asked to record their genuine financial activity for four consecutive weeks.

Instruments. At the end of four weeks each participant completed a structured questionnaire: (i) a 5-point Likert battery covering ease of use, perceived speed, perceived correctness, trust in the multi-user view and willingness to continue; and (ii) four open-ended questions.

5.2 Quantitative Pilot Telemetry

Table 11. *Pilot study aggregate telemetry (four weeks, three families)*

Metric	Value
Ledger entries posted	1 412
Inter-account transfers	87
Bank statement imports	23
Distinct currencies used	4 (EUR, USD, RUB, UZS)
Median session length	2 min 14 s
Median entries per session	3
Sessions initiated while offline	18 %
Operations posted while offline	11 %
Conflicts requiring LWW arbitration	4

Cross-tenant rows leaked (audit log review)	0

The four LWW events were benign: three cases of concurrent metadata edits and one category rename. None resulted in a complaint.

5.3 Qualitative Findings

Table 12. *Likert results from the pilot questionnaire (n=8)*

Dimension	Mean	SD
Ease of use	4.5	0.5
Perceived speed	4.1	0.6
Perceived correctness of the ledger	4.8	0.4
Trust in the multi-user view	4.6	0.5
Willingness to continue using the system	4.3	0.7

Most-valued features. Five of eight participants named the offline outbox as the single most valuable feature: "I posted three expenses on the metro and they were just there when I came up." Two named the multi-currency transfer with automatic round-trip; one named the per-branch role separation.

Most frustrating moments. Three participants cited the bank-statement matcher's first-pass false-negative rate, particularly for non-Latin-script payee names. Two cited the absence of a recurring-entry primitive; one cited the lack of a dark mode for the macOS desktop client.

Most surprising behaviour. Four participants mentioned the system's correct handling of two co-editors posting against the same account simultaneously: the entries appeared in the correct chronological order on both devices within seconds — a directly observable consequence of the HLC-ordered reconciliation.

5.4 Threats to Validity

External validity. A four-week study of three families is not a substitute for a multi-month, multi-cohort field deployment. The eight-participant Likert sample has a 95 % confidence interval of approximately ± 0.7, comparable to inter-item differences.

Selection bias. Participants were recruited from the author's network, were technically literate and knew the author was both researcher and developer. Anonymous administration mitigated but did not eliminate courtesy bias.

Construct validity. Likert items are self-reported perceptions, not observed performance. The quantitative telemetry is the more reliable evidence; Likert ratings are a triangulating signal.

5.5 Discussion

The architectural principles delivered the intended properties. All five principles from Section 2.1 were reflected in observable behaviour: the offline outbox flushed deterministically, the RLS audit produced zero leaks, the UNIQUE constraint absorbed all duplicate-delivery events, and the deterministic LWW resolved all concurrent edits without intervention.

The domain-tailored protocol is the right engineering choice. The Yjs comparison showed a 3.4× storage and 7× CPU advantage for the domain-tailored protocol — consistent with the theoretical argument in Section 1.2.2.

Database-enforced multi-tenancy is qualitatively different. The zero-leak result is structural, not merely quantitative: the absence of any application-layer code path that could leak a row makes the failure-mode unreachable, rather than merely improbable.

The single-author engineering envelope is sufficient. The full system, ~70 000 lines of Dart and 1 800 lines of PL/pgSQL, was implemented and shipped to six platforms by a single author, empirically validating the framework selection decision.

5.6 Chapter Summary

Conclusions for Chapter 5. The pilot study supports the empirical results of Chapter 4 and indicates that the system is fit for its stated purpose with the cohort tested, subject to the threats to validity stated above.

Conclusions and Recommendations

Confirmation of the Research Tasks

The six research tasks formulated in the Introduction have been completed in full:

7. The state of the art has been analysed and a concrete research gap identified (Chapter 1).
8. The application architecture has been designed on the basis of Clean Architecture with a five-principle justification and a documented set of trade-offs (Chapter 2).
9. The PostgreSQL 16 schema and a complete RLS policy set have been specified and audited; the audit returned zero cross-tenant leaks across 5×10^4 requests (Section 4.5).
10. All sixteen functional modules have been implemented in ~70 000 lines of Dart and 1 800 lines of PL/pgSQL and shipped to six target platforms (Chapter 3).
11. The offline-first data layer has been implemented with an HLC-keyed outbox and a single-in-flight reconciliation algorithm; throughput is flat at ≥ 400 ops/s up to $K = 10^4$ queued operations (Section 4.4).
12. The application has been evaluated quantitatively and qualitatively (Chapters 4 and 5).

Validation of the Hypothesis

The hypothesis stated in the Introduction is supported by the evidence presented:

- Effectiveness: interactive p99 latency below 1 s on all three reference devices.
- Transparency: immutable append-only ledger model and per-operation audit trail.
- Collaboration: deterministic HLC-ordered reconciliation under multi-user offline writes.
- Security: zero-leak result of the adversarial RLS audit.
- Multi-currency: commission-aware idempotent atomic transfer primitive.
- Single-author feasibility: the system was built, tested and shipped to six platforms by one engineer within a bachelor's thesis timeframe.

Achievement of the Aim

The aim of the study has been achieved. All quantitative success criteria are satisfied, the qualitative pilot returned a mean Likert score above 4.0 on every dimension,

and the source code is released under the MIT License at https://github.com/farruhsho/ethnocount for academic review.

Scientific Contribution

13. A domain-tailored HLC-keyed offline-first reconciliation protocol for append-mostly accounting ledgers that matches the guarantees of a general-purpose CRDT at 3.4× less storage and 7× less CPU.
14. A static declarative mapping between Clean-Architecture aggregates and PostgreSQL RLS policies, such that cross-tenant leakage is structurally precluded by the query plan.
15. A commission-aware multi-currency atomic transfer primitive, idempotent on a client-supplied operation UUID, preserving the double-entry invariant under banker's rounding, with O(1) complexity per call.

Practical and Theoretical Value

The practical value is the open-source release of a six-platform, multi-tenant, offline-first financial management system. The theoretical value is the demonstration that a domain-tailored synchronization protocol can match the guarantees of a general-purpose CRDT at a fraction of the cost on a workload class (append-mostly ledger) characteristic of accounting, audit and event-sourcing systems generally.

Limitations and Future Work

- The pilot study covered three families over four weeks — not a substitute for a longitudinal multi-cohort field deployment.
- The bank-statement matcher's first-pass accuracy is sensitive to non-Latin-script payee names.
- The system does not currently include recurring entries, Open Banking PSD2 integration, AI categorization or tax computation.

Future work naturally extends the thesis along four directions:

- A multi-month multi-cohort field study ($n \geq 50$ families).
- Integration of an on-device AI categorization model.
- Open Banking PSD2 connectors for the European Economic Area.
- Formalization of the synchronization protocol in TLA$^+$ with a model-checked proof of P_1, P_2 and P_3.

Closing Remark

Beyond its technical contribution, the thesis demonstrates that a single bachelor-level engineer, working with contemporary open-source tooling and a managed Backend-as-a-Service platform, can deliver a production-grade cross-platform application that addresses a real and widespread need in family and small-business financial management.

Literature

[1] Actual Budget Contributors. Actual Budget: a local-first personal finance app. 2024. URL: https://actualbudget.org/

[2] J. F. Bovens. Firefly III: a free and open-source personal finances manager. 2024. URL: https://www.firefly-iii.org/

[3] Firebase Community. Aggregated post-mortems of Firestore Security Rules incidents. 2024. URL: https://github.com/firebase/firebase-tools/issues

[4] Google. Cloud Firestore Security Rules reference. 2024. URL: https://firebase.google.com/docs/firestore/security/get-started

[5] Google. Flutter Documentation. 2024. URL: https://docs.flutter.dev/

[6] D. Hood. Building Progressive Web Apps: Bringing the Power of Native to the Browser. O'Reilly Media, 2019.

[7] C. T. Horngren, W. T. Harrison, M. S. Oliver. Accounting, 10th ed. Pearson, 2015.

[8] K. Jahns. Yjs: a CRDT framework for shared editing. 2020. URL: https://github.com/yjs/yjs

[9] M. Kleppmann, A. Wiggins, P. van Hardenberg, M. McGranaghan. Local-First Software: You Own Your Data, in spite of the Cloud. Onward! Essays, 2018, pp. 154–178.

[10] S. S. Kulkarni, M. Demirbas, D. Madappa, B. Avva, M. Leone. Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases. Tech. Rep. 2014-04, State University of New York at Buffalo, 2014.

[11] A. Larsson, M. Petersson, J. Lind. Tooling-related financial disputes in multi-adult households: a cross-sectional study. Journal of Family and Economic Issues, vol. 44, no. 3, pp. 412–429, 2023.

[12] K. C. Laudon, J. P. Laudon. Management Information Systems: Managing the Digital Firm, 14th ed. Pearson, 2015.

[13] R. C. Martin. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall, 2017.

[14] M. E. Porter. The Five Competitive Forces That Shape Strategy. Harvard Business Review, vol. 86, no. 1, pp. 78–93, 2008.

[15] The PostgreSQL Global Development Group. PostgreSQL 16 Documentation. 2024. URL: https://www.postgresql.org/docs/16/

[16] M. Shapiro, N. Preguiça, C. Baquero, M. Zawirski. Conflict-free Replicated Data Types. Proc. 13th Int. Symp. on Stabilization, Safety, and Security of Distributed Systems (SSS), 2011, pp. 386–400.

[17] Statista Research Department. Number of users of personal finance and banking applications worldwide. 2024.

[18] Supabase Inc. Supabase Documentation. 2024. URL: https://supabase.com/docs

Annex A. Database Schema (DDL Excerpt)

This appendix reproduces the core data definition language for the Ethnocount schema. The complete migration set is published with the source code at https://github.com/farruhsho/ethnocount.

Listing A.1. *Branches and membership tables*

```
CREATE TABLE branches (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        TEXT NOT NULL,
  base_ccy    CHAR(3) NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  created_by  UUID NOT NULL REFERENCES auth.users(id)
);

CREATE TYPE branch_role AS ENUM ('owner','accountant','viewer');

CREATE TABLE branch_members (
  branch_id  UUID NOT NULL REFERENCES branches(id) ON DELETE CASCADE,
  user_id    UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  role       branch_role NOT NULL,
  joined_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  PRIMARY KEY (branch_id, user_id)
);

CREATE INDEX branch_members_user_idx ON branch_members(user_id);
```

Listing A.2. *Ledger entries (append-mostly aggregate)*

```
CREATE TYPE ledger_side AS ENUM ('debit','credit');

CREATE TABLE ledger_entries (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id    UUID NOT NULL REFERENCES branches(id),
  account_id   UUID NOT NULL,
  ts           TIMESTAMPTZ NOT NULL,
  amount_minor BIGINT NOT NULL,
  ccy          CHAR(3) NOT NULL,
  side         ledger_side NOT NULL,
  note         TEXT,
  op_uuid      UUID NOT NULL,
  created_by   UUID NOT NULL REFERENCES auth.users(id),
  hlc          BYTEA NOT NULL,
  CONSTRAINT ledger_op_unique UNIQUE (op_uuid)
);
```

```
CREATE INDEX ledger_branch_ts_idx ON ledger_entries(branch_id, ts DESC);
CREATE INDEX ledger_account_ts_idx ON ledger_entries(account_id, ts DESC);
```

Listing A.3. *Transfers and exchange rates*

```
CREATE TABLE transfers (
  id               UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  branch_id        UUID NOT NULL REFERENCES branches(id),
  src_account      UUID NOT NULL,
  dst_account      UUID NOT NULL,
  src_minor        BIGINT NOT NULL,
  dst_minor        BIGINT NOT NULL,
  rate             NUMERIC(20,10) NOT NULL,
  commission_minor BIGINT NOT NULL DEFAULT 0,
  ts               TIMESTAMPTZ NOT NULL,
  op_uuid          UUID NOT NULL,
  created_by       UUID NOT NULL REFERENCES auth.users(id),
  CONSTRAINT transfers_op_unique UNIQUE (op_uuid)
);

CREATE TABLE exchange_rates (
  branch_id UUID NOT NULL REFERENCES branches(id),
  base_ccy  CHAR(3) NOT NULL,
  quote_ccy CHAR(3) NOT NULL,
  rate      NUMERIC(20,10) NOT NULL,
  as_of     DATE NOT NULL,
  PRIMARY KEY (branch_id, base_ccy, quote_ccy, as_of)
);
```

Listing A.4. *Audit log*

```
CREATE TABLE audit_log (
  id          BIGSERIAL PRIMARY KEY,
  ts          TIMESTAMPTZ NOT NULL DEFAULT now(),
  actor       UUID NOT NULL,
  entity      TEXT NOT NULL,
  entity_id   UUID NOT NULL,
  action      TEXT NOT NULL,
  payload     JSONB
);

CREATE INDEX audit_entity_idx ON audit_log(entity, entity_id);
CREATE INDEX audit_actor_ts_idx ON audit_log(actor, ts DESC);
```

Annex B. System Architecture Diagrams

B.1 Component View

The deployed component view of Ethnocount is described textually below (the LaTeX source in the accompanying package renders the same diagram via TikZ):

```
+---------------------+
|   Flutter UI        |
|   (Widgets)         |
+----------+----------+
           |
           v
+---------------------+
|   BLoC / Cubit      |
+----------+----------+
           |
           v
+---------------------+
|   Use Cases         |
|   (domain)          |
+----------+----------+
           |
           v
+---------------------+
|   Repositories      |
+----+-----------+----+
     |           |
     v           v
+---------+   +---------+
|  Hive   |   | Outbox  |
| (local) |   |         |
+---------+   +----+----+
     ^             |
     |             v
     |        +-------------------+
     +--------+ Supabase          |
   Realtime   | PostgreSQL + RLS  |
   push       | Realtime          |
              +-------------------+
```

Figure 1. *Deployed component view of Ethnocount*

B.2 Offline Write Sequence

Sequence of an offline-first write through to server acknowledgement. The user-visible latency is the time from the tap to step 6 (ok, instant). The network round-trip happens later and is invisible to the user.

```
1. User  → UI       : tap 'Save'
2. UI    → BLoC     : post(entry)
3. BLoC  → Repo     : call use case
4. Repo  → Local    : upsert
5. Local → Repo     : ok
6. Repo  → Outbox   : enqueue(op)
7. Repo  → UI       : ok (instant)
8. Outbox → Supabase : flush (when online)
9. Supabase → Outbox : ack (UNIQUE op_uuid)
```

Figure 2. *Sequence of an offline-first write through to server acknowledgement*

Apliecinājums / Affirmation

Ar šo es, Farruhs Muzrabovs, apliecinu, ka bakalaura darbs ir izpildīts patstāvīgi, bez citu palīdzības, no svešiem avotiem ņemtie dati un definējumi ir uzrādīti darbā. Šis darbs nekādā veidā nav iesniegts nevienai citai pārbaudījuma komisijai un nekur nav publicēts.

(Hereby I, Farruh Muzrabov, affirm that the Bachelor's thesis was performed independently; sources of data and definitions are provided. This work has not been submitted to any other examination commission and has not been published elsewhere.)

Rīga, 20____. _____

_____

paraksts / signature *atšifrējums / decryption*

Pateicības / Acknowledgments

Autors izsaka pateicību darba vadītājam Dr.sc.ing., prof. Andrejam Bondarenko par metodisko vadību, vērtīgajiem ieteikumiem un nepārtraukto atbalstu visā darba izstrādes laikā. Pateicība arī Rīgas Nordic Universitātes Dabaszinātņu un datortehnoloģiju katedras kolēģiem un mācībspēkiem par radīto akadēmisko vidi, kā arī trīm ģimenēm, kas brīvprātīgi piedalījās četru nedēļu pilotpētījumā un sniedza nenovērtējamu praktisko atgriezenisko saiti.

The author expresses sincere gratitude to the supervisor, Dr.sc.ing., Prof. Andrejs Bondarenko, for methodical guidance, valuable advice and continuous support throughout the development of this work. Thanks are also due to the colleagues and academic staff of the Department of Natural Sciences and Computer Engineering at Riga Nordic University for the academic environment, and to the three families who voluntarily took part in the four-week pilot study and provided invaluable practical feedback.